Model Predictive Control: Mini-Project

Tim KOBLER (344385)
Noé SYFRIG (344312)
Marwane MROUEH (356471)

December 2025

Contents

1 Introduction **2**

2 Linearization **2**
2.1 Deliverable 2.1 2

3 Nominal MPC Controller **3**
3.1 MPC Controller for Stabilization (Deliverable 3.1) 3
3.2 MPC Controllers for Tracking (Deliverable 3.2) 6
3.3 PID Position Tracking Controller (Deliverable 3.3) 7

4 Simulation with Nonlinear Rocket **8**
4.1 Deliverable 4.1 8

5 Offset-free Tracking **10**
5.1 Invariant Rocket Mass (Deliverable 5.1) 10
5.2 Changing Rocket Mass (Deliverable 5.2) 12

6 Robust Tube MPC for landing **13**
6.1 Disturbance Rejection on Z (Deliverable 6.1) 13
6.2 Full Disturbance Rejection (Deliverable 6.2) 16

7 NMPC **17**
7.1 Deliverable 7.1 17

1 Introduction

This report covers our development of different MPC controllers to land a rocket (modeled with drone motors), simulating trust vector control. The answer to all deliverables required can be found in this report, and to ease access to them, numeration of deliverable and section of this document are equivalent.

Important remark: the code for our project can be found in the attached ".zip" document.

2 Linearization

2.1 Deliverable 2.1

We can separate the linearized system about the trim point into four independent subsystems (x, y, z, roll). This is due to two aspects: actuation orthogonality and rocket symmetry.

- **Orthogonal actuators**:
  - δ_1 and δ_2 tilt the thrust vector in two perpendicular planes (y-z and x-z respectively). This prevents cross-coupling between x and y motions.
  - P_{avg} controls only the vertical thrust magnitude (z-direction), remaining independent of tilt angles for small angles, which given the state constraints for this problem, is the case.
  - P_{diff} generates pure roll torque about the body z-axis via counter-rotating propellers without affecting the average trust vector.
- **Symmetry and inertia**: The rocket's cylindrical like symmetry aligns the dynamics with the principal body axes (x_b, y_b, z_b). Indeed, in the referential of the rocket, we can make the assumption of a cylinder (with a diagonal inertia tensor) as model for our rocket's inertia. Thus when a torque is applied, the body rotates along one axis only, and not along multiples. This further allows the decoupling of the subsystems.

Because of the linearization, we must ensure that $|\alpha|, |\beta| \leq 10°$; otherwise, the model loses validity. We also note (as mentioned in the project description) that γ has no impact on the validity of the linearization but only affects the accuracy of the x and y subsystems. The decoupling allows simplifying the controller complexity and tuning of each controller independently. Below are all four subsystems summarized for clarity.

Subsystem	**States / Input**	**Physical Isolation**
x	ω_y, β, v_x, x / δ_2	Pitch-plane tilt; orthogonal to y
y	ω_x, α, v_y, y / δ_1	Yaw-plane tilt; orthogonal to x
z	v_z, z / P_{avg}	Vertical thrust; independent of tilt angles
roll	ω_z, γ / P_{diff}	Roll; no dependence on vertical thrust

Table 1: Decoupled subsystems and their physical justification

3 Nominal MPC Controller

3.1 MPC Controller for Stabilization (Deliverable 3.1)

The MPC optimization problem for each subsystem is in a delta formulation. Indeed, since in our project the nonlinear model was linearized around a steady state point x_s, u_s (trim point of the rocket), we have to change the formulation from a normal MPC to a delta formulation to consider the shift toward the linearization point. This is also justified by the matrices A and B both requiring perturbations from the linearization point as inputs.

$$\begin{aligned} \min \quad & \sum_{k=0}^{N-1} (x_k - x_s)^\top Q (x_k - x_s) + (u_k - u_s)^\top R (u_k - u_s) + (x_N - x_s)^\top Q_f (x_N - x_s) \\ \text{s.t.} \quad & x_{k+1} = A(x_k - x_s) + B(u_k - u_s) + x_s \\ & x_0 = x, x_k \in \mathbb{X},\, u_k \in \mathbb{U} \\ & x_N - x_s \in \mathcal{X}_f \end{aligned} \tag{1}$$

In our code we solve directly for x_k. This explains the constraints on the non-shifted state and input. More explanations on the delta formulation are available in the section Delivrable_3_2.

As we learned in class, the controller computes at each step the best trajectory to follow based on the current measured state, and then applies only the first element of this trajectory in a receding horizon fashion. To ensure recursive feasibility, we must define a terminal set $\mathcal{X}_f$. The terminal set is control invariant under the control law $u = Kx$, and a practical choice for this is to use infinite horizon LQR, giving us the control gain K, and terminal weight Q_f by solving the DARE. We made the choice to use IH-LQR for the terminal set for (almost) all of our controllers in this project.

Using LQR as terminal terminal set, This MPC formulation is recursively feasible and stable because:

1. States $x_1^*, \ldots, x_N^*$ already satisfy all constraints from the previous solution
2. $\mathcal{X}_f$ is control invariant under $u = Kx$: $x_N^* \in \mathcal{X}_f \implies x_{N+1} = Ax_N^* + BKx_N^* \in \mathcal{X}_f$
3. The terminal controller $u = Kx$ keeps the system inside $\mathcal{X}_f$ while satisfying all constraints: $x \in \mathbb{X} = \{x | H_x \leq h_x\}$ and $u \in \mathbb{U} = \{u | H_u K x \leq h_u\}$ for all $x \in \mathcal{X}_f$
4. The stage cost function is positive definite, and only zero at origin
5. The terminal cost is a continuous Lyapunov function in the terminal set $\mathcal{X}_f$

Therefore, if the MPC problem enters the terminal set it stays feasible at any time, and stability is also assured.

Since the system can be decomposed into 4 subsystems, we separated the controller into 4 optimization problems for which we defined the constraints, the LQR controller, the invariant set, the cost function, and the terminal cost for each.

3.1.1 Choice of Tuning Parameters

A first approach for our tuning was achieved with the normalization approach for LQR: knowing that $\|u_j\| \leq u_{j,max},\ j = 1, \ldots, m$ and $\|x_i\| \leq x_{i,max},\ i = 1, \ldots, n$, we can define $q_i = \frac{\tilde{q}_i}{x_{i,max}^2}$ and

$r_j = \frac{\tilde{r}_j}{u_{j,max}^2}$. The cost becomes:

$$J = \sum_{k=0}^{+\infty} \frac{\tilde{q}_1}{x_{1,max}^2} x_1^2(k) + \cdots + \frac{\tilde{q}_n}{x_{n,max}^2} x_n^2(k) + \frac{\tilde{r}_1}{u_{1,max}^2} u_1^2(k) + \cdots + \frac{\tilde{r}_m}{u_{m,max}^2} u_m^2(k)$$

and $\tilde{q}_i, \tilde{r}_i$ can be chosen in the interval $(0, 1)$, independently of the measurement units (Taken from Prof. Giancarlo Ferrari Trecate's ME-422 Multivariable Control course, Lecture 8: Optimal control)

However, this approach is not sufficient, and looking at the graphs produced, the necessary weights were further tuned experimentally, sometimes to substancially different values. In our case:

- A large Q enables a fast and efficient reaction on the associated states
- The x and y controllers place high importance on the target angular velocity (with a high q_1 value) and slow down overly aggressive actuator reactions (with a large R) to avoid oscillations
- The Z controller is also aggressive but remains reasonable to avoid exceeding the constraints on P_{avg}
- The time-step was chosen as indicated in the assignment
- Attention was also given to the terminal sets produced with Q and R (using dlqr), so that they are not too small, so as to ensure a good region of attraction, allowing us to decrease H and thus save computation time

More generally, larger $Q \succeq 0$ emphasizes tracking accuracy while larger $R \succ 0$ penalizes control effort. The aggressiveness of the controller is given by the difference between Q and R. This difference is what impacts the cost of the optimization problem.

It is worth to mention that we observed that the controller fails to reach its target if the time horizon H is too small because the optimization problem has no solution. Indeed, if the time horizon is very small, the controller may not be able to reach the terminal set in so few steps, and thus the problem is infeasible. We defined $H = 3$s because it is one of the smallest values for which the problem has a solution, and the larger H is, the more time the controller takes to compute.

3.1.2 Terminal invariant set design

The terminal invariant set $\mathcal{X}_f$ is computed iteratively:

$$\mathcal{X}_f^{i+1} = \{x \in \mathcal{X}_f^i \mid A_{cl}x \in \mathcal{X}_f^i\}, \quad A_{cl} = A + BK, \tag{2}$$

starting from $\mathcal{X}_f^0 = \mathbb{X}_{delta} \cap K\mathbb{U}_{delta}$ with $K\mathbb{U}_{delta} = \{x \mid Kx \in \mathbb{U}_{delta}\}$ and where $\mathbb{X}_{delta} = \{H_x \Delta x \leq h_x - H_x x_s\}$ and $\mathbb{U}_{delta} = \{H_u \Delta u \leq h_u - H_u u_s\}$, until convergence $\mathcal{X}_f^{i+1} = \mathcal{X}_f^i$.

There are few parameters that have influence on the invariant set.

- **State constraints**: Tighter constraints directly limit the invariant set size
- **Input constraints**: These fundamentally limit the achievable invariant set through $\mathcal{KU}$

- **Q matrix**: Higher Q makes the LQR gain more aggressive, shrinking the invariant set but improving tracking.
- **R matrix**: Higher R reduces control effort, enlarging the invariant set but slowing response. Smaller R has a similar effect that a high Q

It is important to note that for the z-controller and roll-controller, we have no state constraints. We could therefore set the terminal set to $\mathbb{R}$ and require $u_N - u_s = 0$, or compute the maximum invariant set of $K\mathbb{U}$ with the IH-LQR controller
However, we decided to do a practical MPC approach and thus ignore the terminal set constraint of these two states, while still using a terminal cost Q_f designed on the LQR. The reason for this is that our initial state will always be in a sufficiently small subset of the feasible set. After testing and chose a proper N (sufficiently large), we came to the conclusion that it works reliably.

On figure 1, 2, 3 and 4 you can observe the terminal invariant set for each dimension.

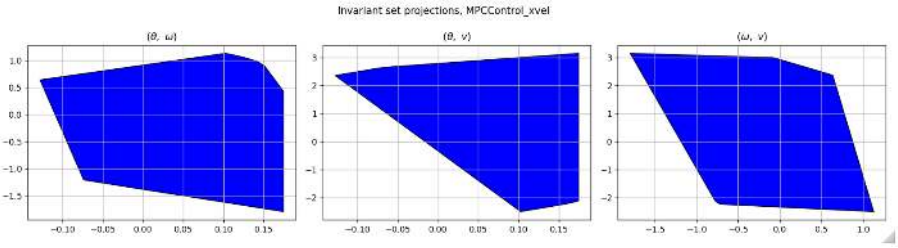

Figure 1: Terminal invariant set of x

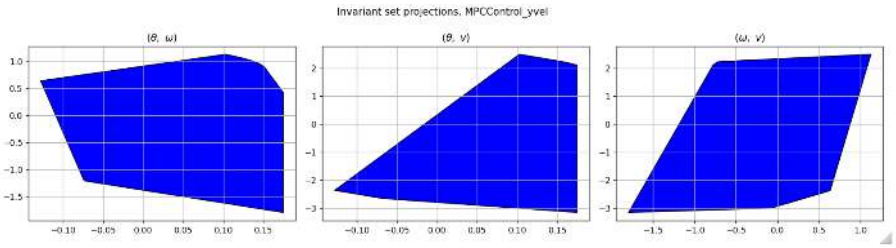

Figure 2: Terminal invariant set of y

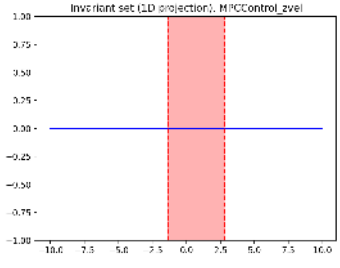

Figure 3: Terminal invariant set of z. Notice that for Z, the invariant set is in 1D (along the blue line, between the red vertical lines).

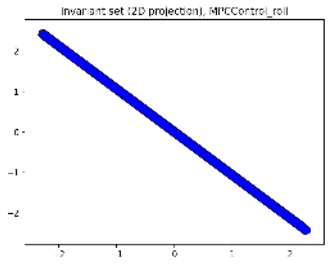

Figure 4: Terminal invariant set of z and roll.

3.1.3 States plot in Open-loop and Closed-loop

On figure 5 and 6 you can observe the system response in open and closed loop using the MPC controller we designed for this section. The starting point is 5 m/s for each dimension (for x, y and z) AND 40deg for roll (γ).

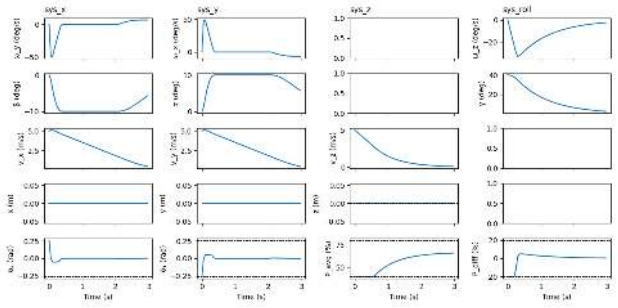

Figure 5: Open-loop plots of the system

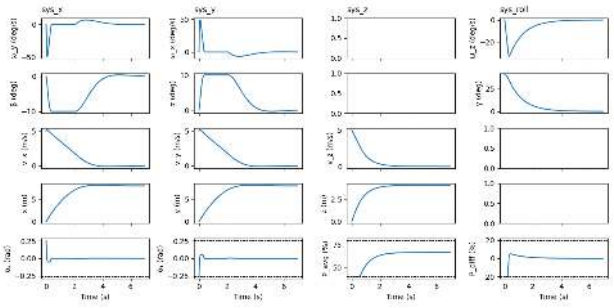

Figure 6: Closed-loop plots of the system

3.2 MPC Controllers for Tracking (Deliverable 3.2)

3.2.1 Design Procedure

The MPC controller of deliverable 3.1 has been extended to track non-zero references by modifying the optimization problem into a delta formulation with $\Delta x_k = (x_k - x_s) - (x_{ss} - x_s) = x_k - x_{ss}$ and $\Delta u_k = (u_k - u_s) - (u_{ss} - u_s) = u_k - u_{ss}$, where x_{ss} and u_{ss} are the target state and input respectively, and x_s, u_s are the trim/linearization values. In the code, we decided to keep the explicit formulation of Δx and thus solved directly for x_k, which slightly modifies the formulation of the problem as presented in class. However, both formulations are equivalent and are summarized below for clarity. We implemented the formulation on the right.

$$
\left\{
\begin{aligned}
\min \; & \sum_{k=0}^{N-1} (\Delta x_k^\top Q \Delta x_k + \Delta u_k^\top R \Delta u_k) \\
& + \Delta x_N^\top Q_f \Delta x_N \\
\text{s.t. } & \Delta x_0 = \Delta x \\
& \Delta x_{k+1} = A \Delta x_k + B \Delta u_k \\
& H_x \Delta x_k \leq k_x - H_x x_{ss} \\
& H_u \Delta u_k \leq k_u - H_u u_{ss} \\
& \Delta x_N = x_N - x_{ss} \in \mathcal{X}_f
\end{aligned}
\right\}
\Leftrightarrow
\left\{
\begin{aligned}
\min \; & \sum_{k=0}^{N-1} (x_k - x_{ss})^\top Q (x_k - x_{ss}) + (u_k - u_{ss})^\top R (u_k - u_{ss}) \\
& + (x_N - x_{ss})^\top Q_f (x_N - x_{ss}) \\
\text{s.t. } & x_0 = x \\
& x_{k+1} = A(x_k - x_{ss}) + B(u_k - u_{ss}) + x_{ss} \\
& H_x x_k \leq k_x, \\
& H_u u_k \leq k_u \\
& x_N - x_s \in \mathcal{X}_f
\end{aligned}
\right\}
$$

To go from the left formulation to the right one, we substituted $\Delta x_k = x_k - x_{ss}$ (resp. $\Delta u_k = u_k - u_{ss}$). Notice that x_s and u_s do not appear on the right, as they cancel out. This is further motivated by the fact that the matrices A and B are for the linearized system, and thus, their inputs must be perturbations from the linearization point. By doing $A[(x_k - x_s) - (x_{ss} - x_s)] = A(x_k - x_s) - A(x_{ss} - x_s)$, and similarly for B, we ensure this condition is respected.

This formulation transforms the problem into regulation around the target steady-state. The terminal invariant sets $\mathcal{X}_f$ and terminal cost Q_f are computed for the regulation case, and taken for the tracking, because the reference is not known a priori. By doing so, $\mathcal{X}_f$ is not the maximum terminal set anymore, and must fullfill the assumption that $x_k \in \mathbb{X}, \forall x_k \in \mathcal{X}_f \oplus x_s$

and $K(x_k - x_s) + u_s \in \mathbb{U}, \forall x_k \in \mathcal{X}_f \oplus x_s$, which we checked for all controlers (even for later deliverables). Therefore, we did not implement any way of rescaling the sets to fulfill the conditions. With this choice, we ensure recursive feasibility and stability around the new equilibrium.

Implementation Details :
The definition of the steady state is the following:

$$\begin{bmatrix} x_{ss} \\ u_{ss} \end{bmatrix} = \begin{bmatrix} A - I & B \\ C & 0 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ r \end{bmatrix} \tag{3}$$

Where in our case $C = I$, since our system is observable. The steady state can be set as input and position target since our case we want to track a velocity reference of 3 m/s (for x, y and z) and to 35°, and the constraints of our system are not related to the velocity or the position. Therefore, (x_s,u_s) satisfy our state and input constraints and can thus be reached, allowing system to converge towards it, and our optimal cost to converge towards 0.

3.2.2 Tuning Parameters

The same tuning parameters from Deliverable 3.1 (Q,R and H) are maintained as they also work well in this configuration.

3.2.3 States plot in Open-loop and Closed-loop

On figure 7 and 8 you can observe the system response in open and closed loop using the MPC controller we designed for this section.

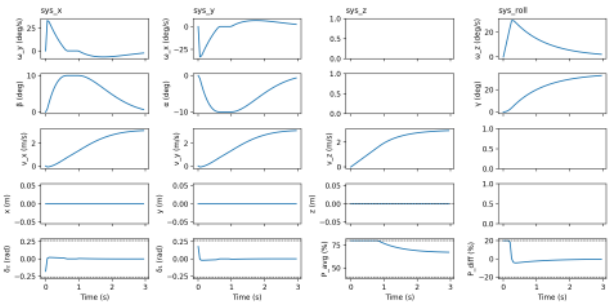

Figure 7: Open-loop plots of the system

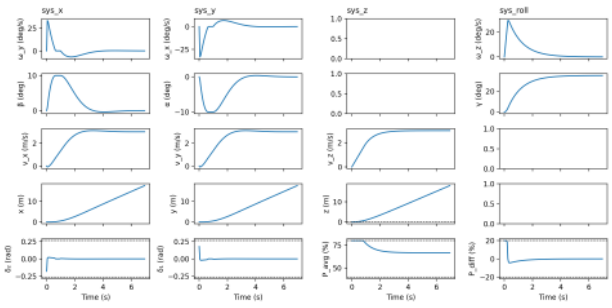

Figure 8: Closed-loop plots of the system

3.3 PID Position Tracking Controller (Deliverable 3.3)

3.3.1 Design procedure

Position tracking requires a hierarchical control structure since we have velocity controllers. The approach is:

1. **Outer Loop (Position):** Compute desired velocities based on position error (PI)
2. **Inner Loop (Velocity):** Use velocity MPC controllers from Deliverable 3.2

As the PI position controller has already been implemented, and the link with the rest of the code also been made (the PI "get_u(...)" is called from the "rocket.simulate_control(...)"

function), we simply used the working velocity controller from Deliverable 3.2, adjusting the tuning to make it work. The PI controller handles slow position dynamics, while the MPC manages fast velocity regulation and constraint satisfaction. This separation simplifies tuning and improves robustness.

3.3.2 Tuning of the Velocity Controller

For the LQR parameters, we started with the values from the deliverable 3.2, however these parameters induced high oscillation for some states (for example some velocities). We therefore increased the respective weights q_i which are directly related to the states, and the increased the weight r_i which are directly related to the inputs affecting the concerned state, until we found a balance. Q and R on x and y to smooth out the trajectory, knowing the target in advance.
We had to take a slightly longer horizon H (6s instead of 3s) to make the problem feasible, at the expense of more computational power. We also increased the simulation time so we could see the whole trajectory and be sure that the rocket reached the target properly.

3.3.3 Closed-loop Plots

On figure 9 you can observe the closed-loop plots starting at $\mathbf{pos} = [50,\ 50,\ 100]\,\mathrm{m}$ tracking a position reference at $\mathbf{pos} = [0,\ 0,\ 10]\,\mathrm{m}$ and a roll angle of $0°$.

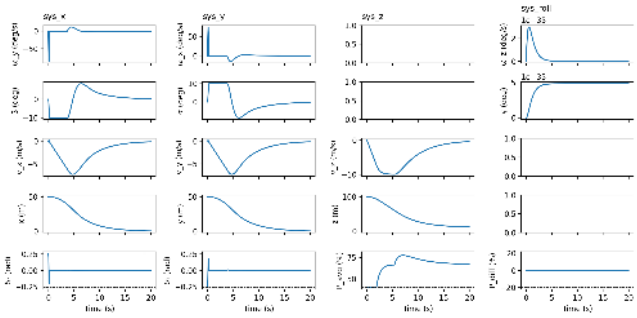

Figure 9: Closed-loop plots of the system

4 Simulation with Nonlinear Rocket

4.1 Deliverable 4.1

For this section, we added to our controller some soft constraints to help with feasibility. Indeed, when we ran the code without the soft constraints, we quickly ran into feasibility issues due to state constraints. To achieve constraints softening, we implemented the simplified algorithm presented in class, with a separation of the objectives. We are aware that by doing so, we create a second independent optimization problem, which would make more sense if we would have to prioritize constraints violations. The advantage however, is that our system will violate the constraints only when absolutely necessary, but this could also have been achieved with a combination of quadratic and linear penalty with the other method. We implemented the soft constraints for the states of the controller x and y, as the other controllers have no

constraints on their state, it is not needed.

Minimization of constraint violation: We first minimize the violation of the state constraints over the prediction horizon:

$$\begin{aligned} \varepsilon^{\min} = \min\{u_k, \varepsilon_k\} \quad & \sum_{k=0}^{N-1} \left(\varepsilon_k^\top S \varepsilon_k + s^\top \varepsilon_k\right) \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k,\ Fx_k \leq f + \varepsilon_k,\ Mu_k \leq m,\ \varepsilon_k \geq 0, \end{aligned} \tag{4}$$

Where the result $\varepsilon_i^{\min}$ denotes the optimal violations obtained from this problem for each of the steps in the horizon of N steps.

Performance optimization: Given the optimal violation levels $\varepsilon_i^{\min}$, we then optimize the controller performance. In the code, we solved directly for x_k, which slightly modifies the formulation of the problem as presented in class, as we mentioned earlier. We now have a systme of the sort:

$$\begin{aligned} \min \quad & \sum_{k=0}^{N-1} (x_k - x_{ss})^\top Q (x_k - x_{ss}) + (u_k - u_{ss})^\top R (u_k - u_{ss}) + (x_N - x_{ss})^\top Q_f (x_N - x_{ss}) \\ \text{s.t.} \quad & x_0 = x \\ & x_{k+1} = A(x_k - x_{ss}) + B(u_k - u_{ss}) + x_{ss} \\ & H_x x_k \leq (k_x + \varepsilon_i^{\min}),\ \text{and}\ H_u u_k \leq k_u \\ & x_N - x_s \in \mathcal{X}_f \end{aligned} \tag{5}$$

We kept the same delta formulation in this deliverable as we did for deliverables 3.2 and 3.3.

4.1.1 Tuning

We took the same tuning as in 3.3 but we slightly adjusted the Q and R of the x and y controller. A higher Q on the angles allows better tracking, and we removed the penalization on the input by setting R to one. The time horizon remains unchanged.

With this, the tracking performance was good.

4.1.2 Closed-loop Plots

On figure 10 you can see the nonlinear rocket successfully performing the maneuver.

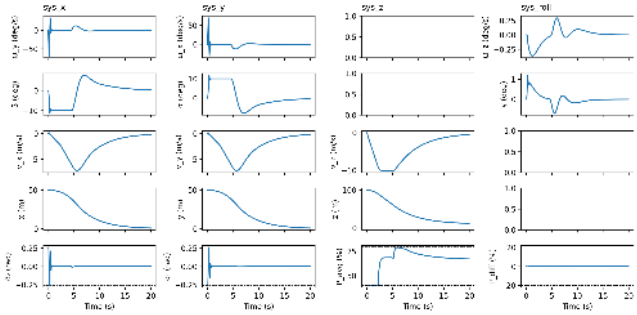

Figure 10: Closed-loop plots of the system

5 Offset-free Tracking

5.1 Invariant Rocket Mass (Deliverable 5.1)

5.1.1 Design Procedure and Tuning

This controller is quite heavily based on the computer exercise 5 and on the structure of the controller 4.1. To allow offset free tracking it includes an observer that estimates the state and the disturbance. This observer is used to adjust the steady-state input target.

Augmented model used for disturbance estimation
Considering the disturbance, we can adapt our model the following way :

$$x_{k+1} = Ax_k + Bu_k + B_d d_k, \tag{6}$$
$$y_k = Cx_k + C_d d_k. \tag{7}$$

$$C = I_1, \quad B_d = B, \quad C_d = 0,$$

the disturbance is modeled as constant: $d_{k+1} = d_k$.

We then apply an observer on an augmented system that combines the state x and the disturbance d.

The linear state estimation can be written as follows:

$$\hat{x}_{k+1} = A_{\text{hat}}\hat{x}_k + B_{\text{hat}}u_k + L\left(\hat{y}_k - y_k\right) \tag{8}$$
$$\hat{y}_k = C_{\text{hat}}\hat{x}_k, \tag{9}$$

with augmented state $\hat{x}_k = \begin{bmatrix} \hat{x_k} \\ \hat{d_k} \end{bmatrix}$ and matrices

$$A_{\text{hat}} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \quad B_{\text{hat}} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \quad C_{\text{hat}} = \begin{bmatrix} C & 0 \end{bmatrix}. \tag{10}$$

Computing steady state targets: To compute the steady-state targets, we start from the augmented steady-state conditions:

$$x_{k+1} = x_k = x_{\text{ss}}, \qquad y_k = r \qquad d_s = \hat{d} = d,$$

with

$$x_{k+1} = Ax_k + Bu_k + B_d d, \qquad y_k = Cx_k + C_d d.$$

At steady state this gives the linear system

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_{\text{ss}} \\ u_{\text{ss}} \end{bmatrix} = \begin{bmatrix} B_d\, d \\ r - C_d\, d \end{bmatrix}. \tag{11}$$

We use $r = x_{\text{target}}$, $C = I$, $C_d = 0$, and $B_d = B$. From the second row of (11):

$$Cx_{\text{ss}} = r \;\Rightarrow\; x_{\text{ss}} = x_{\text{target}}. \tag{12}$$

From the first row of (11):

$$(I - A)x_{\text{ss}} - Bu_{\text{ss}} = Bd \;\Rightarrow\; u_{\text{ss}} = B^{-1}(I - A)x_{\text{ss}} - d. \tag{13}$$

Using (11)'s result $x_{\text{ss}} = x_{\text{target}}$ and the nominal steady-state relation defining u_{target}:

$$Ax_{\text{target}} + Bu_{\text{target}} = x_{\text{target}} \;\Rightarrow\; B^{-1}(I - A)x_{\text{ss}} = u_{\text{target}}, \tag{14}$$

we obtain

$$u_{\text{ss}} = u_{\text{target}} - d. \tag{15}$$

In implementation, the unknown d is replaced by its estimate $\hat{d}$, giving

$$x_{\text{ss}} = x_{\text{target}}, \qquad u_{\text{ss}} = u_{\text{target}} - \hat{d}. \tag{16}$$

Estimation error over time
We have the augmented estimation error and it evolves as:

$$\begin{bmatrix} x_{k+1} - \hat{x}_{k+1} \\ d_{k+1} - \hat{d}_{k+1} \end{bmatrix} = (A_{\text{hat}} + LC_{\text{hat}}) \begin{bmatrix} x_k - \hat{x}_k \\ d_k - \hat{d}_k \end{bmatrix}$$

The error decays exponentially if the eigenvalues of $(A_{\text{hat}} + LC_{\text{hat}})$ are inside the unit circle.

We placed the poles of L at 0.5 and 0.6. The state estimation and disturbance estimation should thus converge.

Tuning
The tuning process uses the basic values from the previous controllers. As in controller 4.1, we increased the time horizon H to make the problem feasible. We chose Q to be large and left R at 1. As the controller was working with these values, we did not tune it further.

5.1.2 Comparison with deliverable 4.1

We observe a clear difference between the nonlinear system with a normal controller (deliverable 4.1) and a controller with offset rejection as implemented in section 5. The most noteworthy changes happen on the z velocity and position, and the Pavg input as you can observe comparing figure 11 and 12. For the controller 4.1, a small offset on the velocity control means that the rocket continues to move up, and thus we notice an important change in z. In the case of controller 5.1 we implemented an offset-free tracking on the velocity controller, taking the offset on the velocity to zero, and thus the position on z stays stable.

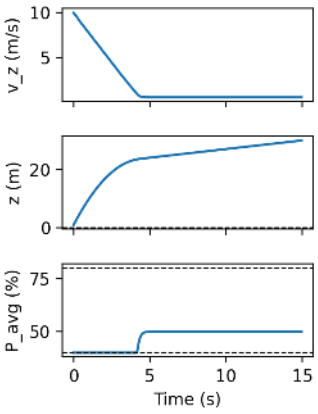

Figure 11: Closed-loop plots of the z controller from part 4.1. We can see the effects of a small v_z offset on the z position.

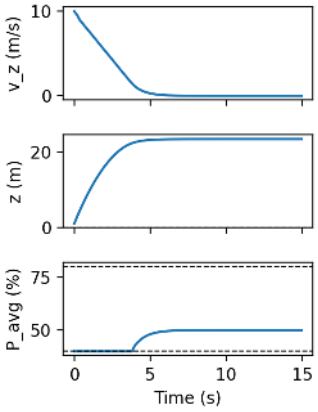

Figure 12: Closed-loop plots of the new offset-free z controller. The offset is compensated.

5.1.3 Disturbance and error

The mass difference is modeled as an unknown constant input disturbance:

$$x_{k+1} = Ax_k + Bu_k + B_d d, \qquad d_{k+1} = d_k. \tag{17}$$

The augmented observer updates $\begin{bmatrix} \hat{x} \\ \hat{d} \end{bmatrix}$ using the innovation $(y_k - \hat{y}_k)$, so the disturbance estimation error $d - \hat{d}_k$ decays approximately exponentially. After the transient passes, $\hat{d}_k$ converges near a constant value, and the controller cancels the offset by using $u_{\text{ss}} = u_{\text{target}} - \hat{d}$.

If the real disturbance is truly constant (due to changing the mass to a different constant value), then $\hat{d}_k$ converges to that constant and tracking becomes offset-free. If the mass decreases due to fuel consumption, $\hat{d}_k$ will vary..

5.2 Changing Rocket Mass (Deliverable 5.2)

In figure 13 we see the plots of the rocket simulation with a varying mass due to fuel consumption.

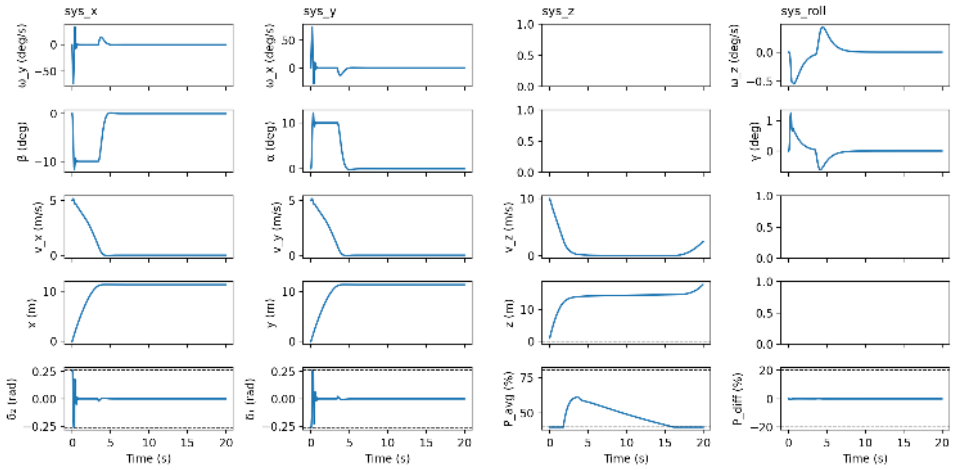

Figure 13: Closed-loop plots of the system with mass variation

5.2.1 Initial Tracking Offset

During the first couple of seconds, there is still an offset in tracking. This is mainly because the disturbance estimator is initialized at $\hat{d}_0 = 0$ and needs a transient to converge, so the controller initially uses an imperfect compensation

$$u_{\text{ss}} = u_{\text{target}} - \hat{d}.$$

Furthermore, when the rocket mass decreases, the disturbance is not actually constant: it changes over time. The estimator is therefore always chasing a moving target, which leaves a tracking offset, seen as a slight ramp in z position and a small offset in z velocity.

To achieve offset-free tracking also for the changing-mass case, the estimator model could be adapted to a time-varying disturbance with for example a ramp model:

$$d_{k+1} = d_k + v_k, \qquad v_{k+1} = v_k,$$

so the observer can track a drifting disturbance.

A simpler improvement is to place faster observer poles (more aggressive gain), but this increases noise sensitivity and can reduce robustness.

5.2.2 Trajectory Characteristics

Throughout the simulation, we may observe:

1. **Initial phase:** Large estimation error as the observer initializes, leading to tracking offset and erratic initial behavior, most easily seen at the very start of the different plots.
2. **Middle phase:** Partial compensation where the estimated disturbance lags behind the true disturbance. This is seen by a slight offset in the z velocity (not exactly 0) and a slight ramp in z position in the middle of the graphs.
3. **Final phase:** Unexpected behavior as the mass approaches limits, and the inputs reaches it's lower bounds.

Late-simulation Behavior At the end of the simulation, we notice unexpected behavior along the z-axis. After reaching the target along the z-axis, there is a point where the rocket begins to regain z velocity, and thus moves upwards. This is due to the rocket loosing to much fuel, and reaching a point where the lower bound of P_{avg} is enough to accelerate the rocket upwards. We see on Figure 13 that the moment the velocity begins to increase again corresponds to the moment P_{avg} hits it's lower bound of 40 and gets stuck there.

6 Robust Tube MPC for landing

6.1 Disturbance Rejection on Z (Deliverable 6.1)

6.1.1 Design Procedure

In this section, we will first consider the general formulation and requirements of robust tube MPC, and then explain the specificity of our implementation. Tube-MPC considers a nominal

system dynamics given by:

$$z_{k+1} = Az_k + Bv_k, \quad k \in \{0, \dots, N-1\},$$

where z_k and v_k denote the nominal state (tube center) and nominal input (tube input), respectively. We compute a stabilizing controller K_s such that our actual noisy system ($x_{k+1} = Ax_k + Bu_k + Bwk$) stays within a radius ε of our tube center, and thus in a set $\mathcal{E}$ around our tube center z_k. To assure that the noisy system and its controller μ_{tube} stay within the constraints ($\mathbb{X}$, resp $\mathbb{U}$), we tighten the constraints on the nominal system for z_k and v_k. For the nominal system to be stable and recursively feasible, we need a terminal set and a terminal constraint given by another control law K_f, and multiple criteria must be fulfilled for the nominal system:

1. The stage cost must be a positive definite function (i.e strictly positive and only zero at the origin)
2. The terminal set must be invariant under the local control law $\kappa_f(z)$, and all tightened state and input constraints must be satisfied in the terminal set $\mathcal{X}_f$
3. The terminal cost must be a continuous Lyapunov function in the terminal set $\mathcal{X}_f$

Note that K_s and K_f could be chosen to be the same, but if so, we lose the benefit of independent tuning of the tube size ($\mathcal{E}$) with the controller K_s and tuning of the terminal set for the tube center with K_f.

The general formulation of tube MPC is therfore the following:

Cost function: The cost function is defined as

$$V(\mathbf{z}, \mathbf{v}) := \sum_{i=0}^{N-1} \ell(z_i, v_i) + V_f(z_N),$$

where $\ell(\cdot, \cdot)$ is the stage cost and $V_f(\cdot)$ is the terminal cost.

Feasible set: The feasible set associated with the initial state x_0 is defined as

$$\mathcal{Z}(x_0) := \left\{ (z, v) \;\middle|\; \begin{array}{ll} z_{i+1} = Az_i + Bv_i, & i \in \{0, \dots, N-1\}, \\ z_i \in \mathbb{X} \ominus \mathcal{E}, & i \in \{0, \dots, N-1\}, \\ v_i \in \mathbb{U} \ominus K_s\mathcal{E}, & i \in \{0, \dots, N-1\}, \\ z_N \in \mathcal{X}_f, & \\ x_0 \in z_0 \oplus \mathcal{E}. & \end{array} \right\}$$

where $\mathcal{E}$ denotes a the minimum robust positively invariant set and $\oplus$ represents the Minkowski sum, and $\ominus$ the Pontryagin difference.

Optimization problem. The Tube-MPC optimization problem is given by

$$\left(v^\star(x_0), z^\star(x_0)\right) = \arg\min_{v,z} \left\{ V(\mathbf{z}, \mathbf{v}) \mid (\mathbf{z}, \mathbf{v}) \in \mathcal{Z}(x_0) \right\}.$$

Control law. The Tube-MPC control law applied to the real system is

$$\mu_{\text{tube}}(x) := K_s\big(x - z_0^\star(x)\big) + v_0^\star(x),$$

where K_s is a stabilizing state-feedback gain and $(z_0^\star, v_0^\star)$ denote the first elements of the optimal nominal state and input sequences.

Our implementation of the tube MPC controller is mainly based on "ex7-sol-tube-mpc.ipynb" of the programming-exercises, however a few differences are to be noted. First our noisy system is of the form: $\Delta \boldsymbol{x_z^+} = A_d \boldsymbol{x_z} + B_d \Delta \boldsymbol{u_z} + B_d w$, where $w \in \mathbb{W} = [-15, 5]$. Once a stabilizing controller K chosen, we can rewrite the system as $\Delta \boldsymbol{x_z^+} = A_{cl} \boldsymbol{x_z} + B_d w$. Because of this, the uncertain state evolution becomes (assuming $\Delta \boldsymbol{x_z}(0) = 0$):

$$\begin{aligned}
\boldsymbol{x_1} &= B_d w_0 \\
\boldsymbol{x_2} &= A_{cl} x_1 + B_d w_1 = A_{cl} B_d w_0 + w_1 \\
&\vdots \\
\boldsymbol{x_i} &= \sum_{k=0}^{i-1} A_{cl}^k B_d w_{i-1-k} = \sum_{k=0}^{i-1} A_{cl}^k B_d w_k
\end{aligned}$$

where $B_d w_k \in S = \{B_d w \mid B_d \cdot (-15) \leq B_d w \leq B_d \cdot 5\} = \left\{ B_d w \;\middle|\; \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} b_{1w} \\ b_{2w} \end{bmatrix} \leq \begin{bmatrix} 5 \\ 5 \\ 15 \\ 15 \end{bmatrix} \right\}$

This new set S must be given to the algorithm for the minimum robust invariant set which was detailed in the lectures.

The other noticeable difference in our implementation is that we use a delta formulation for our robust tube MPC controller. This has as consequence that the state and input constraints are shifted (for example $H_x \Delta x_i \leq h_x - H_x x_s$). This must be accounted for when computing the maximum invariant set, as well as in the optimization problem formulation (referring to section 3, we implemented here the left hand side formulation. This was done so as to shit the focus point from the delta formulation to the tube MPC formulation).

Regarding the choice of the tuning parameters, a similar approach as before was followed. However, it is important to mention that we have $K_s = K_f = K$ in our code. Indeed, tuning two different IH-LQR turned out to create more problems that it solved for us. As a result, the $\mathcal{E}$ tube in our implementation is not as small as it could be, and the controller not as aggressive as it could be, yet it is still meeting the requirements.

6.1.2 Invariant and Terminal Set

You can observe the minimal invariant set $\mathcal{E}$ on figure 14 and the terminal set $\mathcal{X}_f$ on figure 15. The normal input constraints are 40 and 80, while the tightened input constraints $\widetilde{\mathbb{U}}$ are 48 and 59 (non delta formulation). These values are printed in the code, in the z controller.

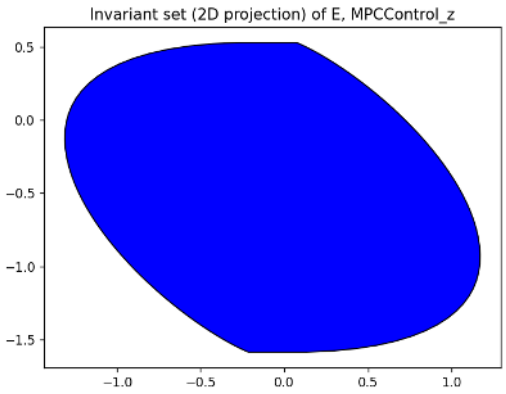

Figure 14: Minimum invariant set $\mathcal{E}$ of the Tube-MPC

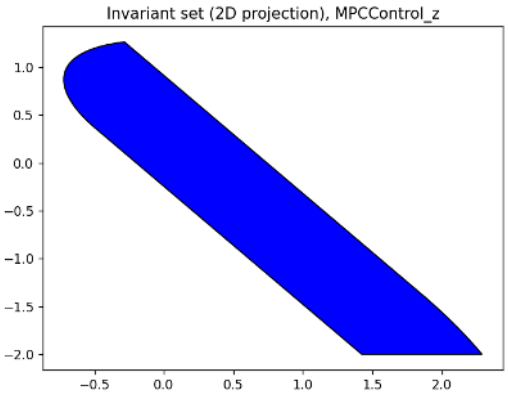

Figure 15: Terminal set $\mathcal{X}_f$ of the Tube-MPC

6.1.3 Closed-loop plots

On figure 16 and 17 you can observe the system response for random and extreme disturbances. We can see that when we introduce noise to our system, we do not converge exactly to the reference anymore, but we stay within an offset. Since we are doing Tube-MPC this is normal because of the size of the $\mathcal{E}$-tube. We notice however, that

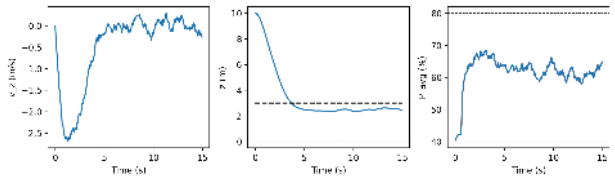

Figure 16: Closed-loop plot for random disturbances

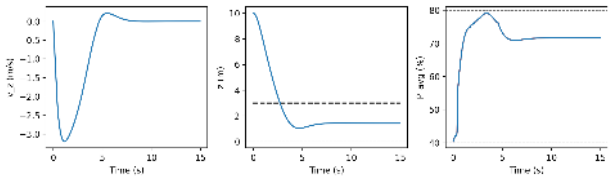

Figure 17: Closed-loop plot for extreme disturbances

6.2 Full Disturbance Rejection (Deliverable 6.2)

6.2.1 Design Procedure and Tuning

For the 3 nominal controller x, y and $roll$, we based ourselves on the ones from the Deliverable 4, that uses delta formulation and soft constraints. We had to modify the x and y controller to include the position on these respective axis. The z controller comes from Deliverable 6.1 and implements a robust Tube-MPC.

Regarding the tuning, we started with the values we had in Deliverable 4. However, the performance of our controller was terrible with high oscillations and long settling time. To solve this issue, we once again increased the corresponding weights in the Q and R matrices and tuned them experimentally until a satisfactory result was obtained.

Regarding the achieved settling time, we based ourselves on the following post of Ed-Discussion: "Tune your controller so that the settling time is around four seconds when landing from z = 10 to z = 3 in the 'no_noise' case. Then use the same controller parameters for the 'random' and 'extreme' disturbance cases.".

6.2.2 Closed-loop plots

On figure 18 you can observe the closed-loop plots showing the performance of the controller.

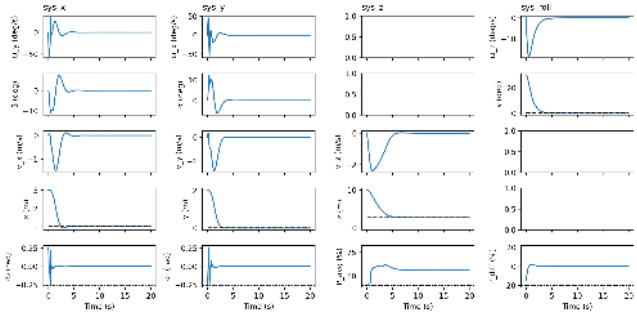

Figure 18: Closed-loop plots of the system (no noise)

7 NMPC

7.1 Deliverable 7.1

7.1.1 Design Procedure and Tuning

The optimization problem can be written as follows:

$$\begin{aligned} \min \quad & \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_i^\top R u_k\right) + x_N^\top Q_f x_N \\ \text{s.t} \quad & x_{k+1} = f(x_k, u_k),\ x_k \in \mathcal{X},\ u_k \in \mathcal{U}, \end{aligned} \tag{18}$$

where $f(\cdot)$ is the nonlinear system dynamics function.

In this section we no longer divide the controller into 4 subsystems, but we manage all the control at once. We added a terminal cost to help reduce the computation time and speed up the simulation process by reducing the time horizon below the ideal value (as indicated in the hint). Constraints on X are reduced significantly because in this controller we only require $x \geq 0$ and $|\beta| \leq 80°$. Additionnally, because $f(\cdot)$ is a continuous function, we have to discretize it at each step. For this we use the Runge-Kutta 4 approach to have better discretization than the Euler approximation. This discretization is then used to update the cost function that we solve with a nonlinear programming approach using Casadi. This process was performed only once during the controller initialization in order to reduce the computational cost. For the implementation of our code, we based ourselves on the programming exercise 9 done in class.

Tuning: The Q matrix is a diagonal matrix with different weights for each of the diagonal elements. We decided to place high importance on the target angle and positions, which will induce a fast response. To attenuate the oscillations on the control inputs in x and y, we increased the factor on λ_1 and λ_2 in the R matrix. This has the effect of slowing down the controller by giving less freedom on the input. The opposite approach was used for roll where we reduced R in order to gain aggressiveness.
As mentioned above, the time horizon H was set to $H = 1$ in order to reduce the computational cost.

7.1.2 Open and Closed-loop Plots

On figure 19 and 20 you can observe the system response in open and closed loop using the MPC controller we designed for this section.

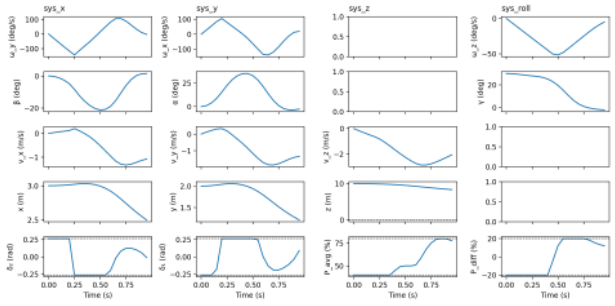

Figure 19: Open-loop plots of the system

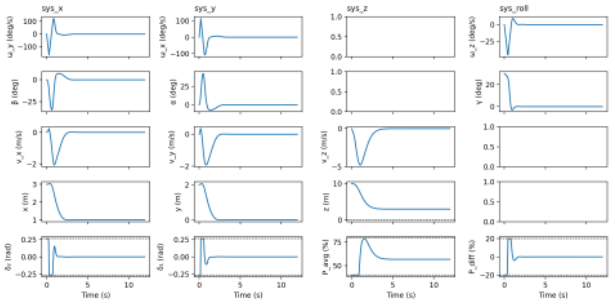

Figure 20: Closed-loop plots of the system

7.1.3 NMPC VS Robust and Nomainal

Nominal MPC is easy to understand and mathematically very strong, it is however constraining due to the terminal set risking being reduced to zero and the invariant region being otherwise very difficult to characterize. For this reason, we applied practical MPC (no terminal set constraint) in our project when it was possible. Up to deliverable 6.2, nominal MPC showed to be an effective ans fast way of controlling our system. For a noisy system, it becomes however more complex and must be combined with observers or robust MPC.

Pros of Robust Linear MPC: The first main advantage is that robust MPC allows for very strong mathematical guarantees (recursive feasibility, stability) under noise. It is therefore easier to analyze and verify because it has explicit robustness to bounded disturbances.
The second big advantage is that it is computationally efficient (QP solving).

Cons of Robust Linear MPC: Despite its strengths, robust linear MPC has several drawbacks: it can be conservative when dealing with large deviations from the linearization point, potentially becoming overly cautious due to its worst-case design approach. It also demands careful design of terminal sets and constraints, often requiring multiple controllers to manage the full system. Furthermore, its performance tends to degrade as uncertainty increases. Since Overall the implementation of Robust Linear MPC such as in section 6.2 was complex, and required a lot of work for it to sucessfully be implemented.

Pros of NMPC: The advantages of an NMPC controller are that we obtain better performance on large maneuvers because we take into account the nonlinear model, which allows for more performant dynamics. It is also easier to design because we design only one single controller (without decomposing the problem into subsystems).

Cons of NMPC: However, NMPC has the disadvantage of being much more complex to solve and that it is not always feasible in real time (more computationally demanding). It is also necessary to have the most accurate nonlinear model possible to best represent reality and thus obtain good results. A final negative point of NMPC is that we have fewer theoretical

guarantees on stability, robustness, as well as optimality (local or global). NMPC is also sensitive to initialization and solver parameters.

Based on the points above and comparing our plots for "robust MPC and three nominal MPCs" (section 6.2, figure 18) and our NMPC controller (figure 20), we can see that the two systems behave a bit differently. However, the difference between the two behaviors is close. Indeed, we can see that the NMPC controller tends to have less smooth curves with more overshoot, but the general form of followed trajectory in closed loop is close. Based on this observation, we can confirm that our linearized model in section 6.2 is behaving as it should, and offers a good approximation and control of the nonlinear behavior of our rocket, allowing us to do an online implementation of the control method.